

Tic Tac Toe (tkinter GUI)

Deep Dive

A clickable 3x3 game window whose rules and unbeatable minimax opponent live in a module that has never heard of a window.

Two Python files, 495 lines of source, zero third-party dependencies.

Written from the source code. Every claim below was read out of the repository or produced by running it.

Contents

1	What this project is	2
1.1	The whole repository	2
1.2	Why it exists, per the README	3
2	Architecture: two layers and a one-way arrow	3
3	game.py: the rules	4
3.1	How a board is represented	4
3.2	WIN_LINES and the bug that shaped it	4
3.3	The rest of the rules	5
4	The AI: minimax	6
4.1	The problem, stated precisely	6
4.2	Scoring a finished game	6
4.3	Minimax proper	7
4.4	Alpha-beta pruning, and what it is not	9
4.5	best_ai_move: the entire public surface of the AI	10
5	main.py: the window	10
5.1	The widget tree	10
5.2	The state the GUI owns	11
5.3	Two independent score dictionaries	12
5.4	The click path, end to end	12
5.5	The 400 ms delay and the event loop	13
5.6	The round as a state machine	14
5.7	_finish_turn_if_over, and the argument nobody reads	14
5.8	Keeping the marks coloured after a cell is disabled	15
5.9	Showing and hiding the mark chooser	15
6	Tests and the unbeatable claim	16
6.1	tests/test_ai.py: six unit tests	16
6.2	tests/verify_unbeatable.py: the actual proof	17
7	Dependencies, licence, and what is deliberately absent	18
8	Honest summary	18

1 What this project is

This is a Tic Tac Toe game you play in a real window with a mouse. You click a square, your mark appears in it, and the program decides whether anyone has won. It has two modes: two people taking turns at the same keyboard, or one person against a computer opponent that cannot be beaten.

That last claim is the interesting part of the project, and it is the reason the codebase is shaped the way it is. “Unbeatable” is not decoration here. It is a specific, mechanically checkable property, and the repository contains the check.

The one idea to take away

The game’s *rules* and the game’s *window* are kept in separate files that know nothing about each other. `game.py` can decide who won without a screen existing; `main.py` draws pixels and never decides anything about the rules. This is why the rules and the AI can be tested by a script with no display attached, which in turn is why the “unbeatable” claim could be verified at all.

1.1 The whole repository

```

tic-tac-toe-gui
├── game.py      154 lines: rules + AI, no I/O
├── main.py      341 lines: the tkinter window
├── tests/
│   ├── test_ai.py      6 unit tests on AI decisions
│   └── verify_unbeatable.py    standalone proof script
├── docs/
│   ├── screenshots/app.png
│   └── explainers/      this document
├── README.md
├── requirements.txt    deliberately empty of packages
├── LICENSE      PolyForm Strict 1.0.0
└── .gitignore

```

Figure 1: Every tracked file. There are only two source files.

Jargon: Module

In Python, a module is simply one `.py` file. Writing `import game` in another file runs `game.py` once and hands you back everything it defined (its functions and variables) so you can call them. “Keeping `game.py` free of tkinter” means: nothing inside that file mentions windows, so importing it never tries to open one.

Jargon: tkinter

The graphical toolkit that ships inside Python itself. It gives you objects for windows, buttons and text labels, and an event loop (see below) that waits for the user to do something. It

needs no installation on Windows or macOS; on Debian and Ubuntu it is a separate system package, `python3-tk`. It is the only reason this project needs anything beyond bare Python, and it is not a third-party dependency: it is part of the standard library.

Jargon: Standard library

The set of modules that arrive with Python itself and need no installation. This project uses exactly three: `tkinter` (the window), `math` (for the values `-inf` and `inf`), and `random` plus `unittest` in the tests. `requirements.txt` therefore lists no packages at all; it contains only a comment saying so.

1.2 Why it exists, per the README

The README states the origin plainly: this began as a “Day 83” console practice script that printed a numbered board and read `input()` from the terminal, and that script had a real bug. Its win check compared cells only against the string “ X ”, so player O could complete a line and the game would carry on as though nothing had happened. The rewrite keeps the original exercise (represent a 3x3 board and detect a win across all eight lines without a library), fixes the bug at its root, and replaces the terminal loop with an actual window.

That history is worth holding onto, because the fix is visible in the current code’s shape. Section 3.2 shows how.

2 Architecture: two layers and a one-way arrow

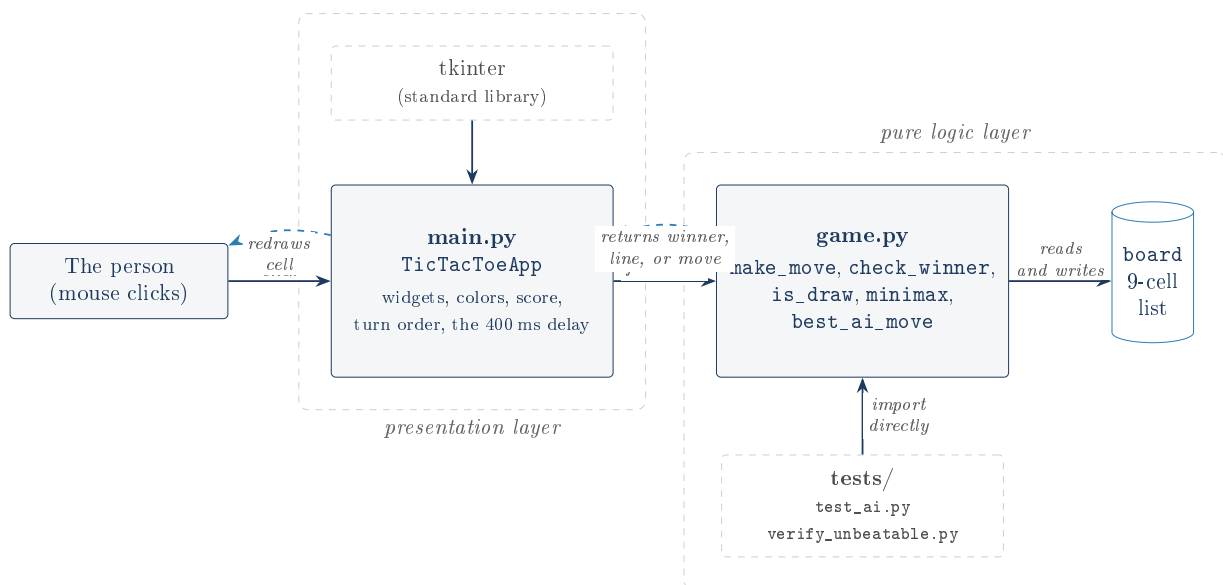


Figure 2: The dependency arrow points one way only. `main.py` imports `game.py`; `game.py` imports nothing but `math`. The tests reach past the window entirely and talk to the logic directly, which is only possible because that arrow never points back.

Why the one-way arrow matters

If `game.py` imported `tkinter`, then `tests/verify_unbeatable.py` could not run: importing the rules would try to touch a display, and on a machine with no screen (a build server, a container) it would fail outright. The 642 exhaustive games in Section 6 were playable precisely because no window was ever involved. The separation is not tidiness for its own

sake; it is what makes the project's headline claim checkable.

3 game.py: the rules

3.1 How a board is represented

The board is a flat Python list of nine strings, not a 3x3 nested list. Each entry is "X", "O", or "" (the constant `EMPTY`). The indices run left to right, top to bottom:

0	1	2
3	4	5
6	7	8

*board[4] is the centre.
board = ["X", "", "O", ...]
is one list of nine strings.*

Figure 3: The flat indexing scheme, copied from the module's own docstring.

Jargon: Flat list vs nested list

A nested list would be `[["X","", ""],["","O",""],["","", ""]]`, addressed as `board[row][col]`. A flat list is `["X","", "", "", "O","", "", "", ""]`, addressed as `board[4]`. Flat wins here because every win line becomes three plain integers, the win table becomes eight tuples of numbers, and the minimax search can place and unplace a move with a single assignment. Nothing in this project ever wants a whole row as an object, so the extra structure would only cost.

3.2 WIN_LINES and the bug that shaped it

There are exactly eight ways to win. The module writes them down once, as data:

```
1 WIN_LINES = [
2     (0, 1, 2), (3, 4, 5), (6, 7, 8),    # rows
3     (0, 3, 6), (1, 4, 7), (2, 5, 8),    # columns
4     (0, 4, 8), (2, 4, 6),              # diagonals
5 ]
```

Listing 1: game.py, the win table

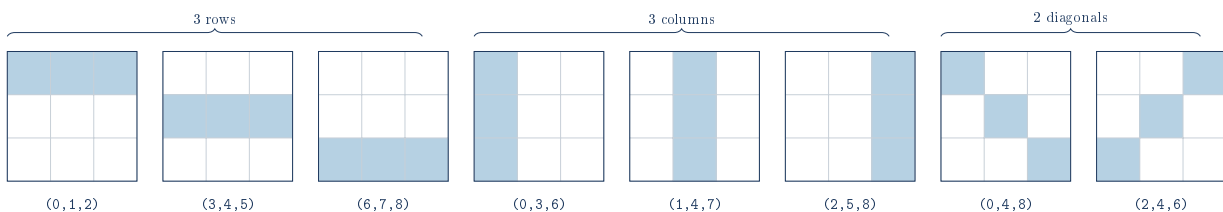


Figure 4: All eight entries of `WIN_LINES`, one board each. Every one is three indices into the flat list, nothing more.

The original console version, per the README, hardcoded eight `or`-chained comparisons and every one of them tested for " X ". Writing the same eight comparisons again for O would have doubled a block that was already hard to read, and it is exactly the kind of block in which one missing case hides. The rewrite sidesteps that by turning the eight lines into a list and looping:

```

1 def check_winner(board):
2     for line in WIN_LINES:
3         a, b, c = line
4         if board[a] != EMPTY and board[a] == board[b] == board[c]:
5             return board[a], line
6     return None

```

Listing 2: game.py, check_winner

The mark is never named in the comparison. The test is “these three cells hold the same thing, and that thing is not empty”, which is true for X and O with one code path. The bug becomes unrepresentable rather than merely fixed.

Data instead of branches

The fix is not “add the missing O check”. It is “stop writing the check per mark”. Eight near-identical `if` statements cannot be audited by eye; eight tuples in a list can be checked against the picture above in seconds. Turning repeated code into a table of data is the move here, and it is the project’s central lesson according to its own README.

The return value is worth noting: `check_winner` gives back the winning mark *and the line*, as a tuple, or `None`. The line is not needed to decide the game; it is needed so the GUI can paint those three cells gold. The rules layer returns it because the presentation layer will want it, but it returns it as three plain integers, not as a colour or a widget. That is the boundary being respected.

3.3 The rest of the rules

Function	Returns	Behaviour
<code>new_board()</code>	a fresh 9-item list	<code>[EMPTY] * 9</code> .
<code>make_move(board, index, mark)</code>	<code>True / False</code>	Writes <code>mark</code> into the cell if the index is in 0..8 <i>and</i> the cell is empty; otherwise changes nothing and returns <code>False</code> . The caller decides what a rejection means.
<code>check_winner(board)</code>	<code>(mark, line)</code> or <code>None</code>	Scans the eight lines, in table order, and stops at the first complete one.
<code>is_draw(board)</code>	<code>True / False</code>	Board full <i>and</i> no winner. Both halves are required: a full board with a winner is a win, not a draw.
<code>is_game_over(board)</code>	<code>True / False</code>	Winner exists, or draw.
<code>available_moves(board)</code>	list of indices	Every empty cell, in board order.
<code>other_mark(mark)</code>	<code>"X" or "O"</code>	<code>"O" if mark == "X" else "X"</code> .

Table 1: The seven public helpers. None of them raise; all of them are total functions over any 9-item list.

`make_move` is the one worth pausing on, because of the order of its test:

```

1 if index < 0 or index > 8 or board[index] != EMPTY:
2     return False

```

Listing 3: game.py, make_move

The range check comes first for a reason: Python’s `or` short-circuits, so if `index` is 12, the expression stops before it ever evaluates `board[12]`, which would raise `IndexError`. Reordering these three conditions would turn a clean `False` into a crash. The guard is correct as written and is fragile to rearrangement, which is the kind of thing worth knowing before touching it.

make_move silently accepts a nonsense mark

Nothing validates that `mark` is "X" or "O". `make_move(board, 0, "banana")` returns `True` and puts the string "banana" on the board, and `check_winner` would happily declare "banana" the winner of a completed line. In practice no caller can do this: both `main.py` and the tests only ever pass marks that came from `other_mark` or a hardcoded literal. So this is a latent sharp edge, not a live bug. It is listed here because a reader auditing `game.py` in isolation should know its guarantees end at the index check.

other_mark treats everything that is not X as O's opposite

`other_mark("banana")` returns "X", because the implementation is a single conditional on `mark == "X"` with no third branch. Same category as above: unreachable given the actual callers, but not a function you would want to expose to untrusted input without a guard.

4 The AI: minimax

This is the part of the project that justifies its existence. Everything above is bookkeeping; this is an actual algorithm.

4.1 The problem, stated precisely

Given a board, and knowing it is my turn, which cell should I play so that I never lose? Not “probably do well” and not “use good heuristics like take the centre”: never lose, against any opponent, ever.

Tic Tac Toe is small enough that this question can be answered by brute force. From an empty board there are at most $9! = 362,880$ orderings of the nine cells, and far fewer real games because most end before the board fills. A computer can look at all of them faster than a human can blink.

Jargon: Game tree

A picture of every way a game can unfold. The root is the position right now. Each branch is one legal move. Each leaf is a finished game (someone won, or the board filled). To “search the game tree” is to walk every branch to its leaf and see what happened.

Jargon: Recursion

A function that calls itself on a smaller version of the same problem. `minimax` asks “how good is this board for me?” by placing each legal move and asking the same question about each resulting board, until it hits a board where the answer is obvious because the game is over. That obvious case is the *base case*, and without one the function would call itself forever.

4.2 Scoring a finished game

At a leaf, the answer is not a guess. It is a fact:

```
1 def _score_terminal(board, ai_mark, depth):
2     result = check_winner(board)
3     if result is None:
4         return 0
5     winner = result[0]
6     return (10 - depth) if winner == ai_mark else (depth - 10)
```

Listing 4: `game.py`, `_score_terminal`

A win for the AI is positive, a loss is negative, a draw is zero. Scores are always from the AI's point of view, no matter whose turn produced the leaf. That single convention is what lets one number flow all the way back up the tree without ever needing to be flipped.

Jargon: depth

How many moves deep in the search this board is. The board handed to `best_ai_move` is depth 0; each move the search tries adds one.

The **depth** term is a tie-breaker. A win at depth 1 scores $10 - 1 = 9$; the same win at depth 5 scores $10 - 5 = 5$. Both are wins, but the shallower one scores higher, so given two winning paths the AI takes the shorter one. It will close out a game it has already won instead of shuffling around the board first. Symmetrically, a loss at depth 7 scores $7 - 10 = -3$, which is less bad than a loss at depth 1 at -9 , so a cornered AI stalls as long as possible.

The losing half of the depth term is dead code in practice

The module's own docstring concedes this: the AI prefers "losing later (if a loss were ever forced, which minimax guarantees it never is against best play)". On a 3x3 board the second player can always force at least a draw, so the AI is never in a genuinely lost position when it is asked to move from a legal game state. The *depth* - 10 branch *does* get evaluated constantly during the search (the search must consider losing lines in order to avoid them), but the tie-break it implements between two losses can never influence the root decision in a real game. It is correct, harmless, and by the code's own admission unreachable in effect.

The magic number 10 is never explained, and does not need to be

It only needs to exceed the maximum depth (9) so that scores never cross zero and change sign: with 10, the worst win still scores $10 - 9 = 1 > 0$ and the mildest loss scores $9 - 10 = -1 < 0$. Had the constant been 8, a win at depth 9 would score -1 and the AI would read some wins as losses. Nothing in the code, the comments or the README states this constraint. It is a load-bearing invariant held together by a bare literal, and there is no test that would catch it being lowered. (The relationship between 10 and the depth bound is my reading of the arithmetic, labelled *inferred*; the code does not say it.)

4.3 Minimax proper

The AI wants a high score; the opponent, playing well, wants a low one. So the search alternates: on the AI's turns take the maximum of the children, on the opponent's turns take the minimum.

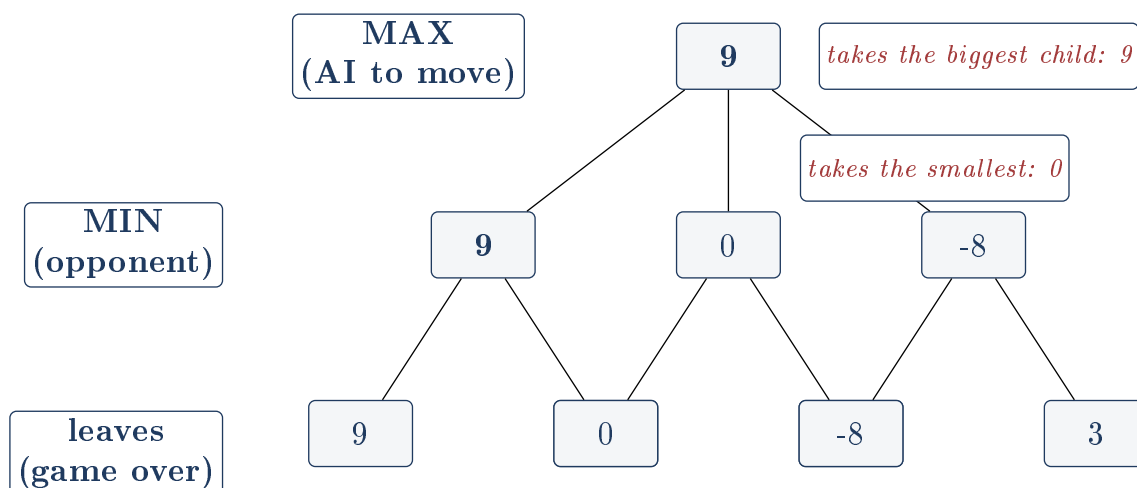


Figure 5: Minimax on a toy tree. The leaf scores come from `_score_terminal`. Each MIN node assumes the opponent will pick the worst outcome for the AI, so it reports the minimum of its children. The root then picks the branch reporting the highest number. Crucially the AI does not hope the opponent blunders: it assumes perfect defence and still finds a line it is happy with. That pessimism is why the guarantee holds against *any* opponent, not just a good one.

```
minimax(board, current_mark, ai_mark, depth, alpha, beta)
```

```

if the game on 'board' is over:
    return score_terminal(board, ai_mark, depth), None
moves := every empty cell
best_move := moves[0]
if current_mark is the AI:
    best_score := -infinity
    for each move in moves:
        place current_mark on board[move]
        score := minimax(board, other(current_mark), ai_mark, depth+1, alpha, beta)
        undo: board[move] := EMPTY
        if score > best_score: best_score, best_move := score, move
        alpha := max(alpha, best_score)
        if beta <= alpha: break // prune
else: // the opponent
    ... identical, but minimising, tracking beta, same prune test
return best_score, best_move

```

Three details in the real code deserve naming.

The board is mutated and restored, not copied. The loop writes `board[move] = current_mark`, recurses, then writes `board[move] = EMPTY` again. A copy per node would be simpler to reason about but would allocate a new list hundreds of thousands of times. The undo makes the function’s contract exact: the docstring promises “`board` is left unchanged on return”, and it is, because every write is paired with its own undo on every path, including the pruning `break` (the undo happens before the comparison that can break).

The undo pattern is correct but unguarded

There is no `try/finally` around the place-recurse-undo sequence. If anything inside the recursion raised, the board would be left with a phantom mark on it and the caller’s board would be silently corrupted. Nothing in this call chain can raise (it is arithmetic and list indexing on a validated structure), so this is theoretical. But the correctness of the “unchanged on return” promise rests on “no exceptions”, not on the language enforcing it.

`best_move` is seeded with `moves[0]`. If every move scored identically, the function still returns

a legal move rather than `None`. This matters because the strict comparison `if score > best_score` never fires on a tie, so without the seed a board where all children tie would return no move at all.

The AI is deterministic. There is no randomness anywhere in `minimax`. Ties are broken by enumeration order, which is board order, which means the first-listed cell achieving the best score wins. On an empty board this produces index 0, the top-left corner, every single time. I confirmed this by running it: `best_ai_move(new_board(), "X")` returns 0, and the full empty-board search completes in about 0.055 seconds on this machine.

A comment in the tests claims the opening is the centre; it is not

`tests/test_ai.py` contains the comment “a center-first opening is the classic optimal choice and is what this specific minimax tie-breaking ... should produce”. It does not. The code returns corner 0, because 0 is enumerated before 4 and both score 0. The comment then partially catches itself in the next sentence and the assertion is loosened to `assertIn(move, [0, 2, 4, 6, 8])`, which passes. So the test is green and its prose is wrong. Verified by running the function directly. Both a corner and the centre are game-theoretically fine openings, so nothing is broken; the comment is just stale reasoning left in the file.

4.4 Alpha-beta pruning, and what it is not

Jargon: Alpha-beta pruning

An optimisation that skips branches which cannot change the answer. `alpha` is the best score the maximiser has already guaranteed itself somewhere else; `beta` is the best the minimiser has guaranteed. When `beta <= alpha`, whatever remains in this branch is unreachable under sensible play by both sides, so the loop stops early.

The intuition, stated as a scenario: you are the maximiser and you have already found a line worth 5. You start exploring another line, and its first reply already drops you to 3. The opponent chooses that reply, so this whole line is worth at most 3. It cannot beat the 5 you already have. There is no reason to look at the rest of it, no matter how good it might get, because you will never be allowed to go there.

Pruning is not why the AI is unbeatable

This distinction is easy to blur and the README explicitly calls it out as a place the author had to correct himself. The unbeatability comes *entirely* from exhaustive minimax search. Delete both `alpha` and `beta` from the code and the AI is exactly as unbeatable, just slower. Conversely, bolt alpha-beta onto a broken search and you get a fast broken search. Pruning changes how much of an already-decided tree gets visited; it never changes the decision. The two claims are independent and should never be merged into “the AI is unbeatable because of alpha-beta minimax”.

Pruning is present but unmeasured, and unnecessary at this size

The code has no benchmark comparing pruned to unpruned, and there is no test that would notice if the pruning were wrong (a bug in the cutoff would show up as a worse move, which the existing tests would catch only by luck). Its practical value here is close to zero: the full empty-board search takes 0.055 seconds *with* pruning, and 3x3 Tic Tac Toe is small enough that the unpruned version would also finish faster than a human notices. Alpha-beta is in this codebase because it is the correct companion to minimax and worth being able to explain, not because the project needed the speed. Saying otherwise would be inventing a rationale

the code does not support.

4.5 best_ai_move: the entire public surface of the AI

```

1 def best_ai_move(board, ai_mark):
2     if is_game_over(board):
3         return None
4     _, move = minimax(board, current_mark=ai_mark, ai_mark=ai_mark)
5     return move

```

Listing 5: game.py, best_ai_move

Four lines. `current_mark` and `ai_mark` are the same at the root, because the root is by definition the AI's turn; they diverge one level down and stay alternating from there. The score is thrown away, because the caller does not want to know how confident the AI is, only where to click.

This is the whole interface `main.py` uses. Give it a board and a mark, get an index back. The GUI has no idea a tree search happened.

5 main.py: the window

5.1 The widget tree

Jargon: Widget

One on-screen thing: a window, a button, a text label, a radio button. In tkinter you create widgets, tell each one who its parent is, and then *pack* it, which asks the parent to find it a place. Widgets form a tree, and the whole window is that tree drawn.

```

root    the Tk window; title "Tic Tac Toe"; not resizable
├── header    Frame
│   └── status_label    "Player X's turn" / "AI is thinking..." / the result
├── mode_frame    Frame
│   ├── Radiobutton "Two player"    writes self.mode
│   └── Radiobutton "Vs AI"    writes self.mode
├── mark_frame    Frame; hidden unless mode is vs_ai
│   ├── Label "You play:"
│   ├── Radiobutton "X (first)"    writes self.human_mark
│   └── Radiobutton "O (second)"
├── grid_frame    Frame
│   └── buttons 0..8    nine Buttons; .grid(row=i//3, col=i%3)
├── controls    Frame
│   └── play_again_btn    calls reset_board
└── score_frame    Frame
    └── score_label    "X wins: 0  O wins: 0  Draws: 0"

```

Figure 6: Built once by `_build_layout`. Every frame is packed vertically; only the nine cells use `.grid`.

Jargon: pack and grid

tkinter's two layout managers. **pack** stacks children in order (top to bottom by default); **grid** places them in rows and columns. They cannot be mixed *inside the same parent*, but different parents may use different ones, which is exactly what happens here: the frames are packed into the window, and the nine buttons are gridded into **grid_frame**.

The nine buttons are created in a loop, and the line that wires them up is the one Python beginners get wrong:

```

1 for i in range(9):
2     row, col = divmod(i, 3)
3     btn = tk.Button(
4         grid_frame, ...,
5         command=lambda idx=i: self.on_cell_click(idx),
6     )
7     btn.grid(row=row, column=col, padx=4, pady=4, ipadx=8, ipady=8)
8     self.buttons.append(btn)

```

Listing 6: main.py, the cell loop

Why lambda idx=i and not lambda: ... (i)

Python closures capture the *variable*, not its value at the time the lambda was written. Writing `lambda: self.on_cell_click(i)` would give all nine buttons a function reading the same `i`, which by the time anyone clicks has finished the loop and equals 8. Every cell would play the bottom-right square. The `idx=i` default argument is evaluated once, when the lambda is created, and freezes the value. This is a genuine bug avoided, not a style choice.

`divmod(i, 3)` returns the quotient and remainder together, so index 5 becomes row 1, column 2. It is the same flat-to-2D mapping the board uses, which is why the button list and the board list can share a single index with no translation anywhere in the file.

5.2 The state the GUI owns

Attribute	Type	Meaning
<code>self.board</code>	9-item list	The only game state. Owned here, operated on by <code>game.py</code> .
<code>self.current_player</code>	"X" / "O"	Whose turn. Always starts at "X".
<code>self.game_over</code>	bool	Set once a round ends; blocks all clicks.
<code>self.ai_thinking</code>	bool	True during the 400 ms pause; blocks all clicks.
<code>self.mode</code>	<code>tk.StringVar</code>	"two_player" or "vs_ai".
<code>self.human_mark</code>	<code>tk.StringVar</code>	"X" or "O"; only read in AI mode.
<code>self.scores_two_player</code>	dict	Keys "X", "O", "draws".
<code>self.scores_vs_ai</code>	dict	Keys "human", "ai", "draws".

Table 2: Note what is absent: no copy of the win lines, no “did X win” flag, no duplicate rule logic. The GUI stores what the GUI needs and asks `game.py` everything else.

Jargon: tk.StringVar

A tkinter box holding a string that widgets can watch. Both radio buttons in a group are handed the same `StringVar` and different values; clicking one writes its value into the box and visually deselects the other. Reading it is `self.mode.get()`, never `self.mode`.

The AI's mark is not stored at all. It is a computed property:

```

1 @property
2 def ai_mark(self):
3     return other_mark(self.human_mark.get())

```

Listing 7: main.py, ai_mark

A stored copy could drift out of sync with the radio button. A property cannot, because there is nothing to keep in sync: it is derived on every read. Small decision, but it is the same instinct as `WIN_LINES`, which is to hold one source of truth and compute the rest.

5.3 Two independent score dictionaries

Two-player scores are keyed by mark ("X", "O"); AI-mode scores are keyed by role ("human", "ai"). This is not an inconsistency, it is the right call: in AI mode the player can switch between X and O between rounds, so a mark-keyed tally would scatter one person's wins across two counters. Keying by role survives the switch. In two-player mode there is no role, only a mark, so the mark is the natural key. Two different questions, two different keys.

The scores vanish when the window closes

They are plain in-memory dictionaries, never written anywhere. Closing the app resets everything to zero. The README's "What I'd do differently" names this explicitly as a known gap, so it is a stated limit rather than an oversight.

5.4 The click path, end to end

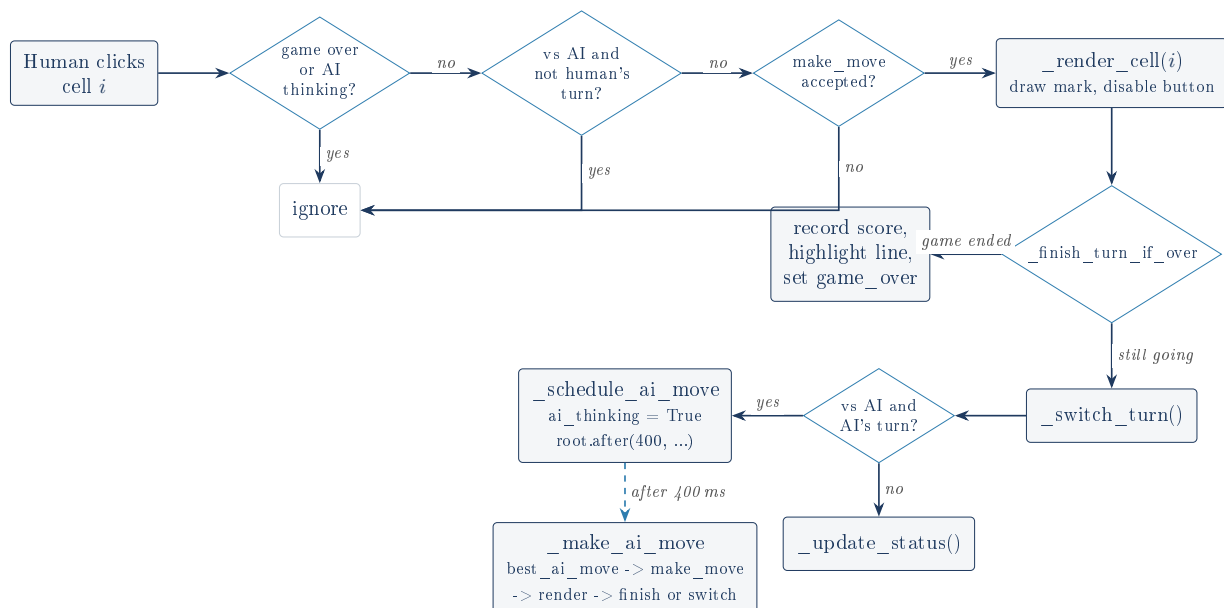


Figure 7: `on_cell_click`, the only entry point a human has into the game. Three guards reject the click before anything happens; the rest is a straight line through `game.py` and back out to the screen.

The three guards deserve enumerating, because between them they cover every way a click can be illegitimate:

1. `self.game_over` or `self.ai_thinking`: the round is finished, or the AI's 400 ms timer is pending. Both are "the board is not yours right now".
2. `mode == "vs_ai"` and `current_player != human_mark.get()`: it is the AI's turn. The comment in the source calls these "stray human clicks".

3. `not make_move(...)`: the cell is already taken. Note the GUI does not check this itself; it lets `game.py` decide and reads the boolean. The rules layer is the authority even on the trivial question.

Guard 3 is doubly covered: filled cells have their button set to `state="disabled"` by `_render_cell`, so tkinter will not even fire the command. The `make_move` check is a second line of defence that would still hold if the disabling were ever removed.

5.5 The 400 ms delay and the event loop

Jargon: Event loop

`root.mainloop()` is a loop that never returns until the window closes. It waits for events (clicks, key presses, timers) and calls your functions in response. Everything your code does happens *inside* a callback from that loop, and while your callback runs, the loop is blocked and the window is frozen.

Jargon: `root.after(ms, fn)`

Asks the event loop to call `fn` once, about `ms` milliseconds from now, and returns immediately. It is not a thread and not a sleep: the window stays responsive in the meantime because control goes straight back to the loop.

The AI's move is computed in about 0.055 seconds in the worst case (empty board, verified by running it), so it would appear instantly. The code inserts a deliberate pause:

```
1 AI_MOVE_DELAY_MS = 400
2
3 def _schedule_ai_move(self):
4     self.ai_thinking = True
5     self._update_status(thinking=True)
6     self.root.after(AI_MOVE_DELAY_MS, self._make_ai_move)
```

Listing 8: main.py, scheduling the AI

The source comment is explicit that this is cosmetic: “The move itself is computed instantly; this delay is purely a UX touch so the AI doesn’t feel like it’s snapping the board shut.” Worth being precise about what it is not: it is not waiting for a computation, and it is not `time.sleep(0.4)`, which would freeze the entire window for 400 ms because the event loop would be blocked inside the callback. Using `after` keeps the window alive; the `ai_thinking` flag is what stops the human from using that liveness to click during the pause.

The `after` handle is discarded, and “Play again” can make the AI move twice

`root.after` returns an id that `root.after_cancel(id)` could use. The return value is thrown away, and `after_cancel` appears nowhere in the file (verified by `grep`: `after` occurs once, as a call).

The consequence is reachable through the UI. Suppose in AI mode the human moves, the AI's timer starts, and 100 ms later the human clicks “Play again”. `reset_board` clears the board and sets `ai_thinking = False` and `game_over = False`, but it cannot stop the pending timer. If the human then plays again, a *second* timer is scheduled while the first is still live. Both fire. Each one calls `_make_ai_move`, which re-checks only `self.game_over` (false) and not whose turn it actually is, so the AI plays a mark, the turn switches to the human, and then the stale callback plays another AI mark out of turn.

The fix is one line at the schedule site (keep the id) and one in `reset_board` (cancel it), or a turn check inside `_make_ai_move`. I could not drive this through the real GUI on this machine

(no display server and no `Xvfb` available), so the failure is traced from the source rather than observed: **labelled inferred**. The discarded return value and the absent `after_cancel` are verified facts; the resulting double move is a reading of the control flow.

5.6 The round as a state machine

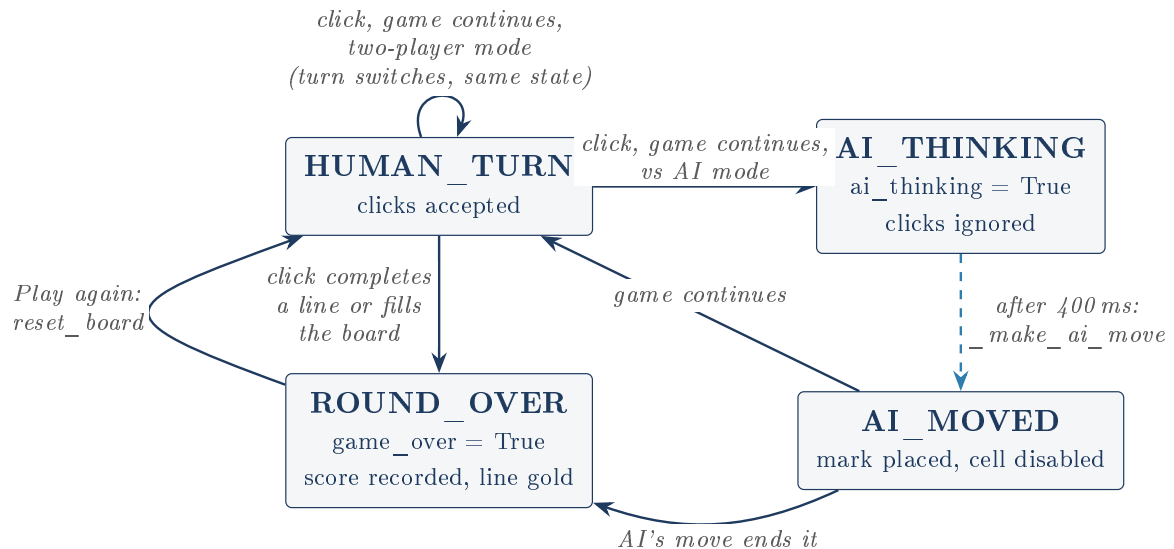


Figure 8: The lifecycle of one round. In two-player mode the `AI_THINKING` and `AI_MOVED` states are never entered; the self-loop on `HUMAN_TURN` is the whole game. `reset_board` is the only edge back out of `ROUND_OVER`, and in AI mode where the AI moves first it fires `_schedule_ai_move` on the way, landing directly in `AI_THINKING` rather than `HUMAN_TURN`.

5.7 `_finish_turn_if_over`, and the argument nobody reads

```

1 def _finish_turn_if_over(self, mark_just_played):
2     result = check_winner(self.board)
3     if result is not None:
4         winner, line = result
5         self.game_over = True
6         self._record_result(winner=winner)
7         self._highlight_line(line)
8         self._update_status(winner=winner)
9         self._update_score_label()
10        return True
11
12    if is_draw(self.board):
13        self.game_over = True
14        self._record_result(draw=True)
15        self._update_status(draw=True)
16        self._update_score_label()
17        return True
18
19    return False

```

Listing 9: `main.py`, end-of-round handling

One function handles both endings, for both players, in both modes, and both call sites (the human's click and the AI's move) use it identically. That is the payoff of the layer split: the GUI's end-of-round handling is five lines of bookkeeping because the hard question was answered elsewhere.

mark_just_played is dead

The parameter is never read in the body. The function asks `check_winner` who won rather than trusting the caller's word for it, which is arguably the better design, but then the argument should have been deleted. Both call sites dutifully pass one (`just_played` from the click handler, `self.ai_mark` from the AI path), and both values are discarded. Verified by reading the body: the only names it touches are `self.board` and its own locals.

5.8 Keeping the marks coloured after a cell is disabled

```
1 self.buttons[index].config(
2     text=mark, fg=color, state="disabled", disabledforeground=color
3 )
```

Listing 10: main.py, `_render_cell`

tkinter greys out a disabled button's text by default, so the blue X and the red O would both fade to the same grey the instant they were placed, and the board would lose its most basic visual signal. Setting `disabledforeground` to the same colour as `fg` defeats that. The README lists this as one of the project's real challenges, and it is a good example of the category: not algorithmically hard, just a toolkit default that silently does the wrong thing.

`_highlight_line` does the same trick in reverse, repainting the three winning cells gold with dark text, again setting `disabledforeground` because those buttons are already disabled by the time anyone has won.

5.9 Showing and hiding the mark chooser

```
1 def _on_mode_change(self):
2     if self.mode.get() == "vs_ai":
3         self.mark_frame.pack(padx=16, pady=(0, 8), fill="x",
4                               before=self._grid_frame())
5     else:
6         self.mark_frame.pack_forget()
7         self._update_score_label()
8         self.reset_board()
9
10 def _grid_frame(self):
11     return self.buttons[0].master
```

Listing 11: main.py, mode switching

`pack_forget` removes a widget from the layout without destroying it, so it can be packed again later with its state intact. But re-packing appends to the bottom of the parent by default, which would drop the mark chooser underneath the board. The `before=` argument pins it back above the grid. `_grid_frame()` recovers that grid frame by asking button 0 who its parent is, since the frame was a local variable in `_build_layout` and was never stored on `self`.

_grid_frame is a workaround for a one-line omission

Storing `self.grid_frame = grid_frame` in `_build_layout` would have made the helper, and its explanatory comment, unnecessary. Reaching through `buttons[0].master` works and is correct, but it re-derives a fact the code already had and then needs a comment to justify itself. Minor, and cosmetic.

Note the side effect: `_on_mode_change` is wired to *all four* radio buttons, including the two mark selectors. So changing your mark mid-game resets the board, with no confirmation. Scores

survive (they are not touched), but the position is gone. Whether that is a bug or the intent is not stated anywhere; it is at least defensible, since changing sides mid-game has no meaning.

6 Tests and the unbeatable claim

6.1 tests/test_ai.py: six unit tests

Jargon: Unit test

A small function that sets up one specific situation, runs one piece of code, and asserts the answer is what it should be. Run them all and you learn whether any of those situations broke. Python's `unittest` module, in the standard library, provides the machinery.

Both test files begin with the same three lines:

```
1 import os, sys
2 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
3 from game import best_ai_move, minimax, new_board
```

This walks up from the test file to the project root and puts it on Python's import path, so `import game` resolves whether you run `python3 tests/test_ai.py` directly or `python3 -m unittest discover -s tests` from the root. There is no `__init__.py` and no packaging; this is the small-project way, and it works.

Test	What it pins down
<code>takes_immediate_winning_move</code>	Board <code>X X _ / 0 0 _ / _ _ _</code> , AI is X: must play 2 and win now.
<code>blocks_immediate_human_win</code>	Board <code>0 0 _ / X _ _ / _ _ _</code> , AI is X: must play 2 to block.
<code>prefers_win_over_block_when_both_available</code>	Claims to test win-over-block. See the finding below.
<code>empty_board_optimal_first_move</code>	Empty board scores 0 (a draw with perfect play) and the move is a corner or the centre.
<code>returns_none_on_full_board</code>	<code>best_ai_move</code> on a full board returns <code>None</code> .
<code>takes_center_when_it_is_the_only_strong_reply</code>	X in a corner, AI is O: must answer 4, the centre.

Table 3: All six pass. Verified: Ran 6 tests ... OK in 0.047 seconds.

Test 3 is a copy of test 1 and tests nothing new

`test_prefers_win_over_block_when_both_available` builds the board `["X", "X", "", "0", "0", "", "", "", ""]` and asserts the move is 2. That is byte-for-byte the same board and the same assertion as `test_takes_immediate_winning_move`. The comment above it is a visible train of thought that never landed: it starts to describe one layout, notices the win cell and the block cell coincide, says “use a layout where winning and blocking are genuinely different cells”, starts a third idea, abandons it with “so build a cleaner case instead.” and then builds the original board again.

So the suite has six tests but five distinct cases, and the specific behaviour this test names (choosing your own win over blocking theirs when they are different cells) is not covered by anything. The comment even flags the trap and the code walks into it anyway. Verified by comparing the two literals directly.

6.2 tests/verify_unbeatable.py: the actual proof

This is not a unit test and does not pretend to be. It is a standalone script that attacks the “unbeatable” claim three different ways, each for both starting orders, and exits non-zero with the losing board if the AI ever loses once.

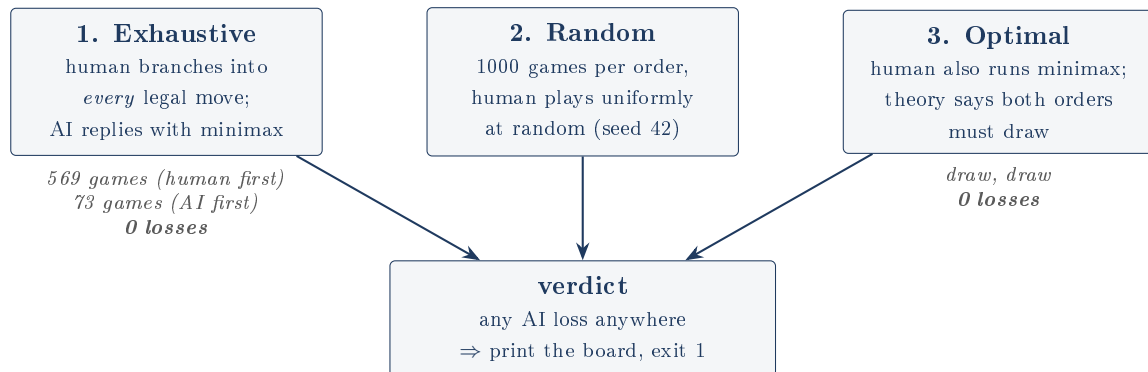


Figure 9: The three checks. Numbers shown are from an actual run on this machine, reproduced below.

Why check 1 is the one that counts

Checks 2 and 3 are supporting evidence. Random play (check 2) can only ever fail to find a counterexample; it samples, so a loss that needs one exact 5-move sequence could hide from a million random games. Check 3 tests a single line of play.

Check 1 is different in kind. It does not sample: at every human turn it recurses into *every* legal move, so it visits every distinct game any human strategy whatsoever could produce against this AI. If the AI could lose, a leaf of that traversal is that loss, and the script finds it. That is the difference between “we did not observe a loss” and “a loss does not exist”. The README’s “What I learned” names this distinction explicitly, and it is the sharpest thinking in the project.

The recursion in check 1 uses the same place-recurse-undo pattern as `minimax`, on its own board, and asymmetrically: the AI’s turn produces one child (minimax is deterministic, so there is exactly one AI reply per position), while the human’s turn produces one child per empty cell. That asymmetry is why the counts are so small. 569 and 73 are not the number of Tic Tac Toe games; they are the number of games reachable *against this particular deterministic opponent*, and the AI-first figure is smaller because the AI’s fixed opening removes a whole layer of human choice before the human ever moves.

Verified output

I ran the script. Every figure the README states is reproduced exactly:

```

1  === 1. Exhaustive search over every human move sequence ===
2  [human moves first] 569 distinct games explored -> AI won: 386, draws: 183, AI lost: 0
3  [AI moves first] 73 distinct games explored -> AI won: 71, draws: 2, AI lost: 0
4
5  === 2. 1000 games, human plays uniformly random legal moves ===
6  [human moves first] 1000 games -> AI won: 789, draws: 211, AI lost: 0
7  [AI moves first] 1000 games -> AI won: 996, draws: 4, AI lost: 0
8
9  === 3. Both sides play optimally (human also uses minimax) ===
10 [human moves first] outcome: draw
11 [AI moves first] outcome: draw
12
13 RESULT: the AI never lost a single game across all checks above.
  
```

Listing 12: python3 tests/verify_unbeatable.py, run on this machine, 57 seconds

Check 2 is seeded (`random.Random(42)`), so those counts are reproducible rather than merely typical, which is why the README can quote 789 and 211 as fixed numbers.

What the verification does not cover

It exercises `game.py` only. Not one line of `main.py` is tested by anything, anywhere in this repository. The AI is proven; the window is not. Every GUI behaviour in this document (the guards, the delay, the highlight, the score dictionaries, the stale-timer bug above) rests on reading the source, because there is no display on this machine and no `Xvfb` to fake one. The project's testing stops precisely at the layer boundary, which is honest about where the interesting logic is, but it does mean the double-move bug in `_schedule_ai_move` could never have been caught by the suite as it stands.

The screenshot in docs/ predates the AI

`docs/screenshots/app.png` shows the status label, the 3x3 grid, “Play again” and an `X wins / 0 wins / Draws` score line, but there are no “Two player” and “Vs AI” radio buttons, and no mark chooser. `_build_layout` packs `mode_frame` with both radios unconditionally, so the current program cannot produce that window. The image documents an earlier build. It is not referenced from the README, so nothing states a false claim; the file is simply stale.

7 Dependencies, licence, and what is deliberately absent

`requirements.txt` contains no packages at all, only a comment explaining that there are none and noting the `python3-tk` caveat on Linux and the Python 3.6+ floor. The floor is real: the code uses f-strings (`f"Player {winner} wins!"`), which arrived in 3.6.

The three imports across the whole project are `math` (only for `math.inf`, the initial alpha and beta), `tkinter`, and `random` plus `unittest` in the tests. Every one ships with Python.

Jargon: `math.inf`

Positive infinity as a float. It seeds the maximiser's `best_score` at `-inf` so that the very first real score, however bad, is an improvement. This avoids needing a separate “have I looked at anything yet” flag.

The licence is PolyForm Strict 1.0.0, copyright 2026 Salman Adnan. Strict means exactly what it says: the code may be read but not used, modified or redistributed. This is a portfolio artefact, not an open-source offering, and the licence matches that intent rather than contradicting it.

`.gitignore` covers the usual Python debris (`__pycache__`/, `*.pyc`, `.pytest_cache/`, `.venv/`, `venv/`) and `.DS_Store`. Note `.pytest_cache/` is listed although the project uses `unittest`, not `pytest`; harmless, and evidently a template line.

8 Honest summary

What this project does well is a single thing done consistently: it draws a line between rules and pixels and never crosses it. That line is not a stylistic preference here. It is load-bearing, because it is what let a script play 642 exhaustive games and 4000 random ones with no window in sight, and therefore what turned “unbeatable” from a boast into a number. The README's own conclusion, that a claim is only worth writing if something behind it would catch the claim being false, is demonstrated by the repository rather than asserted by it.

The AI is a correct, textbook minimax with alpha-beta, and the project is careful in exactly the place most write-ups are careless: it refuses to credit pruning for the guarantee, which belongs to the exhaustive search alone.

Against that, the flaws are real and worth stating without softening:

- **A reachable GUI bug.** The discarded `after` handle lets “Play again” during the AI’s pause leave a live timer that can make the AI move twice (inferred from source; not observed, no display available).
- **A duplicate test.** Six tests, five cases. The one behaviour named in test 3’s own title is not covered by anything, and its comment shows the author spotting the problem and then not fixing it.
- **A wrong comment.** The test suite claims the AI opens in the centre. It opens at corner 0. Verified by running it.
- **A stale screenshot.** `docs/screenshots/app.png` shows a build with no AI mode.
- **Zero coverage of `main.py`.** Every GUI claim in this document is read, not run.
- **Small latent sharp edges.** `make_move` accepts any string as a mark; `other_mark` has no third branch; the constant 10 in `_score_terminal` silently depends on exceeding the maximum depth.

None of these touch the headline claim. The AI is unbeatable, it is verified exhaustively, and I reproduced the verification. The defects live in the layer nobody tested, which is exactly where one would predict them.

Claim in this document	Basis
All README verification numbers (569, 73, 789, 211, 996, 4, zero losses)	Verified: script run, output reproduced above
Six unit tests pass in 0.047s	Verified: <code>unittest discover</code> run
Empty-board opening is corner 0, not the centre	Verified: function called directly
Full empty-board search takes about 0.055s	Verified: timed on this machine
Test 3 duplicates test 1	Verified: literals compared
<code>mark_just_played</code> is unused	Verified: body read
<code>after</code> handle discarded, no <code>after_cancel</code>	Verified: <code>grep</code>
The resulting AI double-move	Inferred: traced through the source; no display available to drive the GUI
The 10 in <code>_score_terminal</code> must exceed max depth	Inferred: my reading of the arithmetic; unstated in the code
Console-version origin and its X-only win bug	From the README; the original script is not in this repository
Screenshot predates the AI mode	Verified: image viewed, no radio buttons present

Table 4: Where each non-obvious claim comes from.